

INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO



Programas Clase 24

Unidad de Aprendizaje: *Sistemas Distribuidos*

Profesor: *Coronilla Contreras Ukranio*

Alumnos:

De la Rosa Gutiérrez Kaleb Yael

Grupo*: 7CM2*

Servidor:

```java
import com.google.gson.Gson;
import com.google.gson.JsonSyntaxException;
import com.sun.net.httpserver.HttpExchange;
import com.sun.net.httpserver.HttpHandler;
import com.sun.net.httpserver.HttpServer;

import java.io.ByteArrayOutputStream;
import java.io.IOException;
import java.io.InputStream;
import java.io.OutputStream;
import java.net.InetSocketAddress;
import java.nio.charset.StandardCharsets;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.Map;
import java.util.concurrent.Executors;

/**
 * Servidor Web RESTful (micro-SAES) en Java con soporte JSON (Google Gson) y Códigos de Estado HTTP.
 *
 * Implementa la especificación completa solicitada en la Clase 24:
 * 1. Endpoint inexistente o incorrecto (/chetito) -> 404 Not Found con JSON de error.
 * 2. Identificador no numérico (GET /alumnos/p, DELETE /alumnos/f) -> 400 Bad Request con {"error": "ID invalido"}.
 * 3. POST /alumnos con JSON de entrada -> 201 Created con JSON del nuevo alumno (incluye ID asignado).
 * 4. GET /alumnos/{id}: si existe -> 200 OK con JSON del alumno; si no existe -> 404 Not Found con {"error": "Alumno no encontrado"}.
 * 5. PUT /alumnos/{id} con JSON de actualización: si existe -> 200 OK con {"mensaje": "Alumno actualizado"}; si no existe -> 404 Not Found con {"error": "Alumno no encontrado"}.
 * 6. DELETE /alumnos/{id}: si existe -> 204 No Content (sin cuerpo); si no existe -> 404 Not Found con {"error": "Alumno no encontrado"}.
 *
 * Mantiene activo el endpoint /status para compatibilidad y health check.
 * Sistemas Distribuidos - ESCOM IPN
 */
public class WebServer {

    private static final int DEFAULT_PORT = 8080;
```

```
    private final int port;
    private HttpServer server;

    // Instancia de Google Gson para serialización y deserialización JSON
    private final Gson gson = new Gson();

    // Estructura de almacenamiento en memoria para los alumnos
    private final Map<Integer, Alumno> alumnos = new HashMap<>();
    private int ultimoId = 0;

    public WebServer(int port) {
        this.port = port;
    }

    public void start() throws IOException {
        server = HttpServer.create(new InetSocketAddress(port), 0);

        // Registro de endpoints del servicio
        server.createContext("/status", new StatusHandler(port));
        server.createContext("/alumnos", new AlumnosHandler());
        server.createContext("/", new RootHandler(port));

        // Pool de hilos para atención concurrente de solicitudes HTTP
        server.setExecutor(Executors.newFixedThreadPool(8));
        server.start();


System.out.println("============================================================
=======");
        System.out.println(" Servidor RESTful micro-SAES (Clase 24 - Gson)
iniciado en: " + port);
        System.out.println(" Endpoint de estado:  http://localhost:" + port +
"/status");
        System.out.println(" Endpoint de alumnos: http://localhost:" + port +
"/alumnos");
        System.out.println(" Formato de datos:    JSON (application/json)");

System.out.println("============================================================
=======");
    }

    public static void main(String[] args) {
```

```
        int port = DEFAULT_PORT;
        if (args.length > 0) {
            try {
                port = Integer.parseInt(args[0]);
            } catch (NumberFormatException e) {
                System.err.println("Puerto inválido proporcionado. Usando
puerto por defecto: " + DEFAULT_PORT);
            }
        }

        try {
            WebServer webServer = new WebServer(port);
            webServer.start();
        } catch (IOException e) {
            System.err.println("Error al iniciar el servidor en el puerto " +
port + ": " + e.getMessage());
        }
    }

    /**
     * Handler para el endpoint /alumnos.
     * Delega el procesamiento al método estructurado handleAlumnosRequest.
     */
    class AlumnosHandler implements HttpHandler {
        @Override
        public void handle(HttpExchange exchange) throws IOException {
            handleAlumnosRequest(exchange);
        }
    }

    /**
     * Método principal de despacho REST para el endpoint /alumnos.
     * Mapea cada método HTTP (GET, POST, PUT, DELETE) con su respectiva
operación y código de estado.
     */
    private void handleAlumnosRequest(HttpExchange exchange) throws
IOException {
        String ruta = exchange.getRequestURI().getPath();
        String metodo = exchange.getRequestMethod().toUpperCase();

System.out.println("-------------------------------------------------------
-------");
```

```
        System.out.println("[PETICIÓN] Método: " + metodo + " | Ruta: " +
ruta);

        // Validar que la ruta sea exactamente /alumnos o empiece con
/alumnos/
        if (!ruta.equals("/alumnos") && !ruta.startsWith("/alumnos/")) {
            System.out.println("[RESPUESTA 404] Endpoint incorrecto: " +
ruta);
            enviarErrorJson(exchange, 404, "Recurso no encontrado");
            return;
        }

        String[] partes = ruta.split("/");
        // ruta = "/alumnos"   -> partes: ["", "alumnos"] (longitud 2)
        // ruta = "/alumnos/1" -> partes: ["", "alumnos", "1"] (longitud 3)

        switch (metodo) {
            case "GET":
                handleGet(exchange, partes);
                break;
            case "POST":
                handlePost(exchange, partes);
                break;
            case "PUT":
                handlePut(exchange, partes);
                break;
            case "DELETE":
                handleDelete(exchange, partes);
                break;
            default:
                System.out.println("[RESPUESTA 405] Método no permitido: " +
metodo);
                enviarErrorJson(exchange, 405, "Método no permitido");
                break;
        }
    }

    /**
     * Procesa peticiones GET:
     * - GET /alumnos       -> Retorna lista completa de alumnos en JSON (200
OK)
```

```
     * - GET /alumnos/{id}  -> Retorna alumno en JSON (200 OK) o 404 Not Found
si no existe.
     * - GET /alumnos/p     -> Retorna 400 Bad Request con {"error": "ID
invalido"} (Inciso 2)
     */
    private void handleGet(HttpExchange exchange, String[] partes) throws
IOException {
        // Caso: GET /alumnos o GET /alumnos/
        if (partes.length == 2 || (partes.length == 3 &&
partes[2].trim().isEmpty())) {
            synchronized (alumnos) {
                String jsonLista = gson.toJson(new
ArrayList<>(alumnos.values()));
                System.out.println("[RESPUESTA 200] Listando " +
alumnos.size() + " alumno(s)");
                enviarRespuestaJson(exchange, 200, jsonLista);
            }
            return;
        }

        // Caso: GET /alumnos/{id}
        if (partes.length == 3) {
            int idBuscado;
            try {
                idBuscado = Integer.parseInt(partes[2].trim());
            } catch (NumberFormatException e) {
                // Inciso 2: Identificador no numérico
                System.out.println("[RESPUESTA 400] Identificador no numérico:
" + partes[2]);
                enviarErrorJson(exchange, 400, "ID invalido");
                return;
            }

            synchronized (alumnos) {
                Alumno alumno = alumnos.get(idBuscado);
                if (alumno != null) {
                    // Inciso 4: Alumno existe -> 200 OK y JSON con datos
                    String json = gson.toJson(alumno);
                    System.out.println("[RESPUESTA 200] Alumno encontrado ID="
+ idBuscado + " | " + json);
                    enviarRespuestaJson(exchange, 200, json);
                } else {
```

```
                    // Inciso 4: Alumno no existe -> 404 Not Found y JSON
{"error": "Alumno no encontrado"}
                    System.out.println("[RESPUESTA 404] Alumno no encontrado
ID=" + idBuscado);
                    enviarErrorJson(exchange, 404, "Alumno no encontrado");
                }
            }
            return;
        }

        System.out.println("[RESPUESTA 404] Ruta inválida bajo /alumnos");
        enviarErrorJson(exchange, 404, "Recurso no encontrado");
    }

    /**
     * Procesa peticiones POST /alumnos:
     * - Lee payload en formato JSON (Inciso 3).
     * - Asigna ID autoincremental.
     * - Guarda en el Map.
     * - Retorna código 201 Created y el JSON del alumno con su nuevo ID
(Inciso 3).
     */
    private void handlePost(HttpExchange exchange, String[] partes) throws
IOException {
        if (partes.length > 2 && !partes[2].trim().isEmpty()) {
            System.out.println("[RESPUESTA 400] POST no admite identificador
en la URI");
            enviarErrorJson(exchange, 400, "POST debe enviarse a /alumnos sin
ID en la ruta");
            return;
        }

        String cuerpo = leerCuerpo(exchange);
        if (cuerpo.isEmpty()) {
            System.out.println("[RESPUESTA 400] POST recibido con cuerpo
vacío");
            enviarErrorJson(exchange, 400, "Cuerpo de solicitud vacío");
            return;
        }

        Alumno nuevoAlumno;
        try {
```

```
            nuevoAlumno = gson.fromJson(cuerpo, Alumno.class);
        } catch (JsonSyntaxException e) {
            System.out.println("[RESPUESTA 400] Sintaxis JSON inválida en
POST: " + e.getMessage());
            enviarErrorJson(exchange, 400, "Formato JSON inválido");
            return;
        }

        if (nuevoAlumno == null || nuevoAlumno.getNombre() == null) {
            System.out.println("[RESPUESTA 400] Datos de alumno incompletos en
POST");
            enviarErrorJson(exchange, 400, "Datos del alumno incompletos");
            return;
        }

        synchronized (alumnos) {
            int nuevoId = ++ultimoId;
            nuevoAlumno.setId(nuevoId);
            alumnos.put(nuevoId, nuevoAlumno);

            // Inciso 3: Respuesta 201 Created con JSON completo
            String jsonCreado = gson.toJson(nuevoAlumno);
            System.out.println("[RESPUESTA 201] Creado nuevo Alumno ID=" +
nuevoId + " | " + jsonCreado);
            enviarRespuestaJson(exchange, 201, jsonCreado);
        }
    }

    /**
     * Procesa peticiones PUT /alumnos/{id}:
     * - Recibe JSON con datos actualizados (Inciso 5).
     * - Si ID no es numérico -> 400 Bad Request {"error": "ID invalido"}
(Inciso 2).
     * - Si alumno no existe -> 404 Not Found {"error": "Alumno no
encontrado"} (Inciso 5).
     * - Si existe y se actualiza -> 200 OK {"mensaje": "Alumno actualizado"}
(Inciso 5).
     */
    private void handlePut(HttpExchange exchange, String[] partes) throws
IOException {
        if (partes.length < 3 || partes[2].trim().isEmpty()) {
            System.out.println("[RESPUESTA 400] PUT sin ID en la ruta");
```

```
            enviarErrorJson(exchange, 400, "ID no proporcionado en la ruta");
            return;
        }

        int idActualizar;
        try {
            idActualizar = Integer.parseInt(partes[2].trim());
        } catch (NumberFormatException e) {
            // Inciso 2: Identificador no numérico
            System.out.println("[RESPUESTA 400] ID no numérico en PUT: " +
partes[2]);
            enviarErrorJson(exchange, 400, "ID invalido");
            return;
        }

        synchronized (alumnos) {
            if (!alumnos.containsKey(idActualizar)) {
                // Inciso 5: Si no existe devuelve 404 y JSON {"error":
"Alumno no encontrado"}
                System.out.println("[RESPUESTA 404] Alumno no encontrado para
actualizar ID=" + idActualizar);
                enviarErrorJson(exchange, 404, "Alumno no encontrado");
                return;
            }

            String cuerpo = leerCuerpo(exchange);
            if (cuerpo.isEmpty()) {
                System.out.println("[RESPUESTA 400] PUT con cuerpo vacío");
                enviarErrorJson(exchange, 400, "Cuerpo de solicitud vacío");
                return;
            }

            Alumno datosActualizados;
            try {
                datosActualizados = gson.fromJson(cuerpo, Alumno.class);
            } catch (JsonSyntaxException e) {
                System.out.println("[RESPUESTA 400] Sintaxis JSON inválida en
PUT: " + e.getMessage());
                enviarErrorJson(exchange, 400, "Formato JSON inválido");
                return;
            }
```

```
            if (datosActualizados == null) {
                System.out.println("[RESPUESTA 400] Datos nulos en PUT");
                enviarErrorJson(exchange, 400, "Datos del alumno inválidos");
                return;
            }

            // Actualizar datos del alumno conservando el ID
            datosActualizados.setId(idActualizar);
            alumnos.put(idActualizar, datosActualizados);

            // Inciso 5: Respuesta 200 OK con {"mensaje": "Alumno
actualizado"}
            Map<String, String> respuestaOk = new HashMap<>();
            respuestaOk.put("mensaje", "Alumno actualizado");
            String jsonOk = gson.toJson(respuestaOk);
            System.out.println("[RESPUESTA 200] Alumno ID=" + idActualizar + "
actualizado | " + jsonOk);
            enviarRespuestaJson(exchange, 200, jsonOk);
        }
    }

    /**
     * Procesa peticiones DELETE /alumnos/{id}:
     * - Si ID no es numérico -> 400 Bad Request {"error": "ID invalido"}
(Inciso 2).
     * - Si alumno no existe -> 404 Not Found {"error": "Alumno no
encontrado"} (Inciso 6).
     * - Si alumno existe -> 204 No Content sin cuerpo (Inciso 6).
     */
    private void handleDelete(HttpExchange exchange, String[] partes) throws
IOException {
        if (partes.length < 3 || partes[2].trim().isEmpty()) {
            System.out.println("[RESPUESTA 400] DELETE sin ID en la ruta");
            enviarErrorJson(exchange, 400, "ID no proporcionado en la ruta");
            return;
        }

        int idBorrar;
        try {
            idBorrar = Integer.parseInt(partes[2].trim());
        } catch (NumberFormatException e) {
            // Inciso 2: Identificador no numérico (ej. DELETE /alumnos/f)
```

```
            System.out.println("[RESPUESTA 400] ID no numérico en DELETE: " +
partes[2]);
            enviarErrorJson(exchange, 400, "ID invalido");
            return;
        }

        synchronized (alumnos) {
            if (!alumnos.containsKey(idBorrar)) {
                // Inciso 6: Si no existe para ser borrado devuelve 404 y JSON
{"error": "Alumno no encontrado"}
                System.out.println("[RESPUESTA 404] Alumno no encontrado para
eliminar ID=" + idBorrar);
                enviarErrorJson(exchange, 404, "Alumno no encontrado");
                return;
            }

            alumnos.remove(idBorrar);
            // Inciso 6: Si la solicitud es correcta devuelve sólo el código
204
            System.out.println("[RESPUESTA 204] Alumno ID=" + idBorrar + "
eliminado exitosamente");
            enviarRespuestaSinCuerpo(exchange, 204);
        }
    }

    /**
     * Lee de forma completa el cuerpo de datos del mensaje HTTP entrante en
UTF-8.
     */
    private String leerCuerpo(HttpExchange exchange) throws IOException {
        InputStream is = exchange.getRequestBody();
        ByteArrayOutputStream buffer = new ByteArrayOutputStream();
        byte[] temp = new byte[1024];
        int leidos;
        while ((leidos = is.read(temp, 0, temp.length)) != -1) {
            buffer.write(temp, 0, leidos);
        }
        return new String(buffer.toByteArray(),
StandardCharsets.UTF_8).trim();
    }

    /**
```

```
     * Envía respuesta HTTP con encabezado Content-Type: application/json;
charset=UTF-8
     * y el cuerpo JSON especificado.
     */
    private void enviarRespuestaJson(HttpExchange exchange, int statusCode,
String jsonResponse) throws IOException {
        byte[] bytesRespuesta = (jsonResponse +
"\n").getBytes(StandardCharsets.UTF_8);
        exchange.getResponseHeaders().set("Content-Type", "application/json;
charset=UTF-8");
        exchange.sendResponseHeaders(statusCode, bytesRespuesta.length);
        try (OutputStream os = exchange.getResponseBody()) {
            os.write(bytesRespuesta);
        }
    }

    /**
     * Envía respuesta HTTP sin cuerpo (ej. 204 No Content).
     */
    private void enviarRespuestaSinCuerpo(HttpExchange exchange, int
statusCode) throws IOException {
        exchange.getResponseHeaders().set("Content-Type", "application/json;
charset=UTF-8");
        exchange.sendResponseHeaders(statusCode, -1);
        exchange.close();
    }

    /**
     * Helper para enviar respuestas de error en formato JSON {"error": "..."}
     */
    private void enviarErrorJson(HttpExchange exchange, int statusCode, String
mensajeError) throws IOException {
        Map<String, String> error = new HashMap<>();
        error.put("error", mensajeError);
        String json = gson.toJson(error);
        enviarRespuestaJson(exchange, statusCode, json);
    }

    /**
     * Handler para el endpoint /status.
     * Mantiene comprobación de disponibilidad activa del servicio.
     */
```

```java
    static class StatusHandler implements HttpHandler {
        private final int port;

        public StatusHandler(int port) {
            this.port = port;
        }

        @Override
        public void handle(HttpExchange exchange) throws IOException {
            String response = "El servidor está vivo en el puerto " + port +
"\n";
            byte[] bytes = response.getBytes(StandardCharsets.UTF_8);
            exchange.getResponseHeaders().set("Content-Type", "text/plain;
charset=UTF-8");
            exchange.sendResponseHeaders(200, bytes.length);
            try (OutputStream os = exchange.getResponseBody()) {
                os.write(bytes);
            }
        }
    }

    /**
     * Handler para rutas no registradas o la raíz "/".
     * Cumple con el Inciso 1: para cualquier endpoint no existente o
incorrecto (ej. /chetito),
     * devuelve el código de estado 404 puesto que ese recurso no existe en el
servidor.
     */
    static class RootHandler implements HttpHandler {
        private final int port;
        private final Gson gson = new Gson();

        public RootHandler(int port) {
            this.port = port;
        }

        @Override
        public void handle(HttpExchange exchange) throws IOException {
            String path = exchange.getRequestURI().getPath();
            String metodo = exchange.getRequestMethod();
```

```
System.out.println("------------------------------------------------------------
-------");
            System.out.println("[ROOT HANDLER] Método: " + metodo + " | Ruta:
" + path);

            // Inciso 1: Si el endpoint no existe o es incorrecto (ej.
/chetito) -> 404 Not Found
            if (!path.equals("/")) {
                System.out.println("[RESPUESTA 404] Endpoint inexistente o
incorrecto: " + path);
                Map<String, String> error = new HashMap<>();
                error.put("error", "Recurso no encontrado");
                byte[] bytes = (gson.toJson(error) +
"\n").getBytes(StandardCharsets.UTF_8);
                exchange.getResponseHeaders().set("Content-Type",
"application/json; charset=UTF-8");
                exchange.sendResponseHeaders(404, bytes.length);
                try (OutputStream os = exchange.getResponseBody()) {
                    os.write(bytes);
                }
                return;
            }

            // Para la raíz "/" informativa
            Map<String, Object> info = new HashMap<>();
            info.put("servicio", "Servidor RESTful micro-SAES (Clase 24)");
            info.put("puerto", port);
            info.put("endpoints_disponibles", new String[]{"/alumnos",
"/status"});
            byte[] bytes = (gson.toJson(info) +
"\n").getBytes(StandardCharsets.UTF_8);
            exchange.getResponseHeaders().set("Content-Type",
"application/json; charset=UTF-8");
            exchange.sendResponseHeaders(200, bytes.length);
            try (OutputStream os = exchange.getResponseBody()) {
                os.write(bytes);
            }
        }
    }
}
```

Alumno.java:

```java
/**
 * Clase Alumno para el servicio REST micro-SAES.
 * Representa la entidad Alumno con identificador único y atributos
académicos.
 * Sistemas Distribuidos - ESCOM IPN
 */
public class Alumno {
    int id;
    String nombre;
    String apellido1;
    String apellido2;
    double promedio;

    public Alumno() {
    }

    public Alumno(int id, String nombre, String apellido1, String apellido2,
double promedio) {
        this.id = id;
        this.nombre = nombre;
        this.apellido1 = apellido1;
        this.apellido2 = apellido2;
        this.promedio = promedio;
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }
```

```
    public String getApellido1() {
        return apellido1;
    }

    public void setApellido1(String apellido1) {
        this.apellido1 = apellido1;
    }

    public String getApellido2() {
        return apellido2;
    }

    public void setApellido2(String apellido2) {
        this.apellido2 = apellido2;
    }

    public double getPromedio() {
        return promedio;
    }

    public void setPromedio(double promedio) {
        this.promedio = promedio;
    }

    @Override
    public String toString() {
        return "ID: " + id + "\tNombre: " + nombre + "\tApellido 1: " +
apellido1 + "\tApellido 2: " + apellido2 + "\tPromedio: " + promedio + "\n";
    }
}
```

GsonAlumnos.java:

```
import com.google.gson.Gson;
import java.util.*;
import com.google.gson.reflect.TypeToken;
import java.lang.reflect.Type;

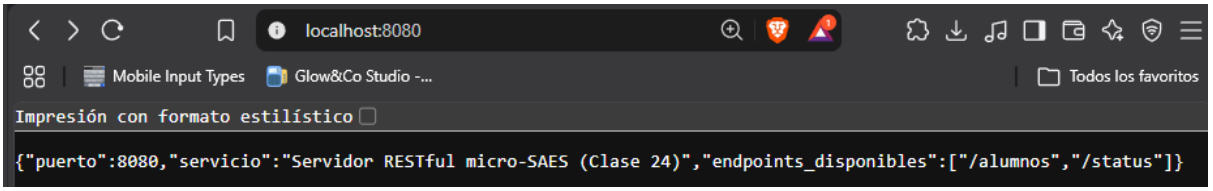
public class GsonAlumno {
```

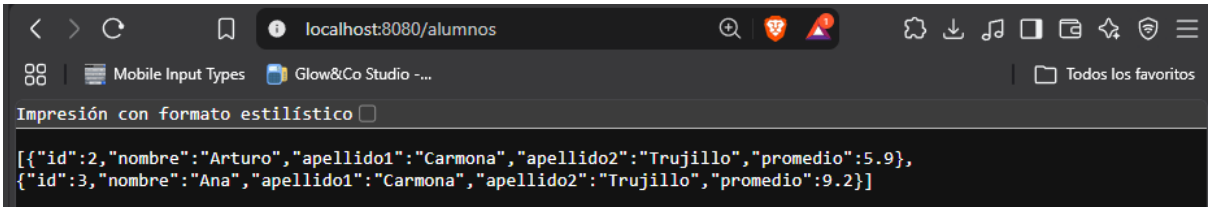
```java
    public static void main(String[] args) {
        // Convertir objeto Alumno a JSON
        Gson gson = new Gson();
        Alumno alumno = new Alumno(1, "Carlos", "Pérez", "Castro", 8.7);
        String jsonAlumno = gson.toJson(alumno);
        System.out.println(jsonAlumno);

        // Convertir JSON a objeto Alumno
        Alumno alumno2 = gson.fromJson(jsonAlumno, Alumno.class);
        System.out.println(alumno2.getNombre());

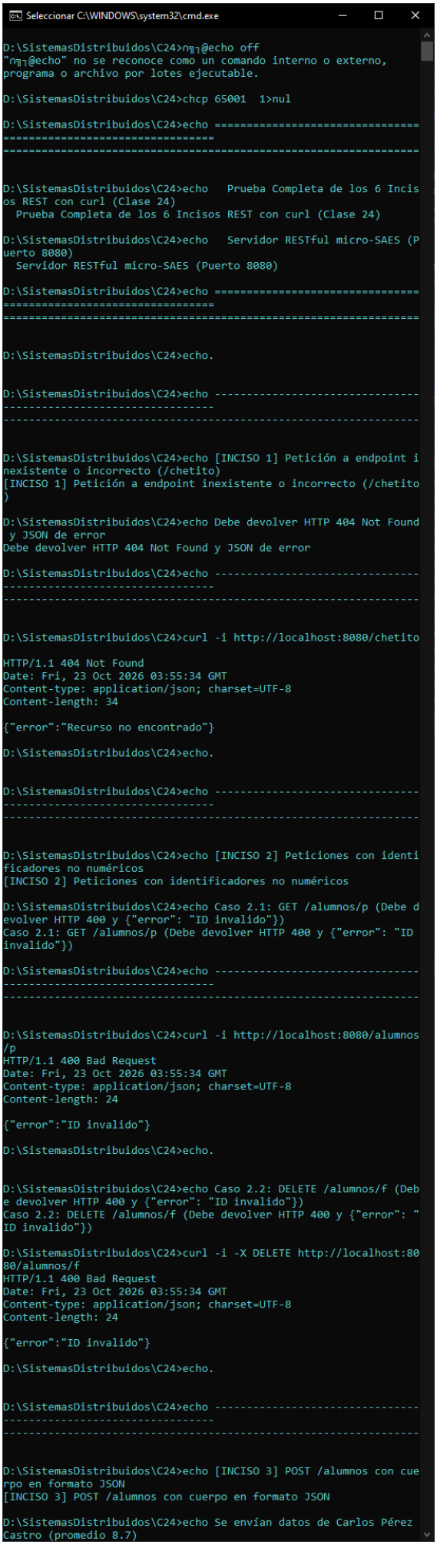
        // Convertir lista de objetos a JSON
        List<Alumno> alumnos = new ArrayList<>();
        alumnos.add(new Alumno(1, "Ana", "Carmona", "Trujillo", 9.1));
        alumnos.add(new Alumno(2, "Luis", "Cuellar", "Paz", 8.5));
        alumnos.add(new Alumno(3, "Carlos", "Pérez", "Castro", 8.7));
        String jsonAlumnos = gson.toJson(alumnos);
        System.out.println(jsonAlumnos);

        // Convertir JSON a lista de objetos
        Type tipoLista = new TypeToken<List<Alumno>>(){}.getType();
        List<Alumno> alumnos2 = gson.fromJson(jsonAlumnos, tipoLista);
        for (Alumno a : alumnos2) {
            System.out.println(a.getNombre());
        }
    }
}
```


localhost:8080
Mobile Input Types
Glow&Co Studio -...
Todos los favoritos
Impresión con formato estilístico
{"puerto":8080,"servicio":"Servidor RESTful micro-SAES (Clase 24)","endpoints_disponibles":["/alumnos","/status"]}


localhost:8080/alumnos
Mobile Input Types
Glow&Co Studio -...
Todos los favoritos
Impresión con formato estilístico
[{"id":2,"nombre":"Arturo","apellido1":"Carmona","apellido2":"Trujillo","promedio":5.9},
{"id":3,"nombre":"Ana","apellido1":"Carmona","apellido2":"Trujillo","promedio":9.2}]


localhost:8080/status
Mobile Input Types
Glow&Co Studio -...
Todos los favoritos
El servidor está vivo en el puerto 8080


Seleccionar C:\WINDOWS\system32\cmd.exe
D:\SistemasDistribuidos\C24>กๆๅ@echo off
"กๆๅ@echo" no se reconoce como un comando interno o externo,
programa o archivo por lotes ejecutable.
D:\SistemasDistribuidos\C24>chcp 65001 1>nul
D:\SistemasDistribuidos\C24>echo ==========
==========
==========
D:\SistemasDistribuidos\C24>echo Prueba Completa de los 6 Incisos REST con curl (Clase 24)
Prueba Completa de los 6 Incisos REST con curl (Clase 24)
D:\SistemasDistribuidos\C24>echo Servidor RESTful micro-SAES (Puerto 8080)
Servidor RESTful micro-SAES (Puerto 8080)
D:\SistemasDistribuidos\C24>echo ==========
==========
==========
D:\SistemasDistribuidos\C24>echo.
D:\SistemasDistribuidos\C24>echo ----------

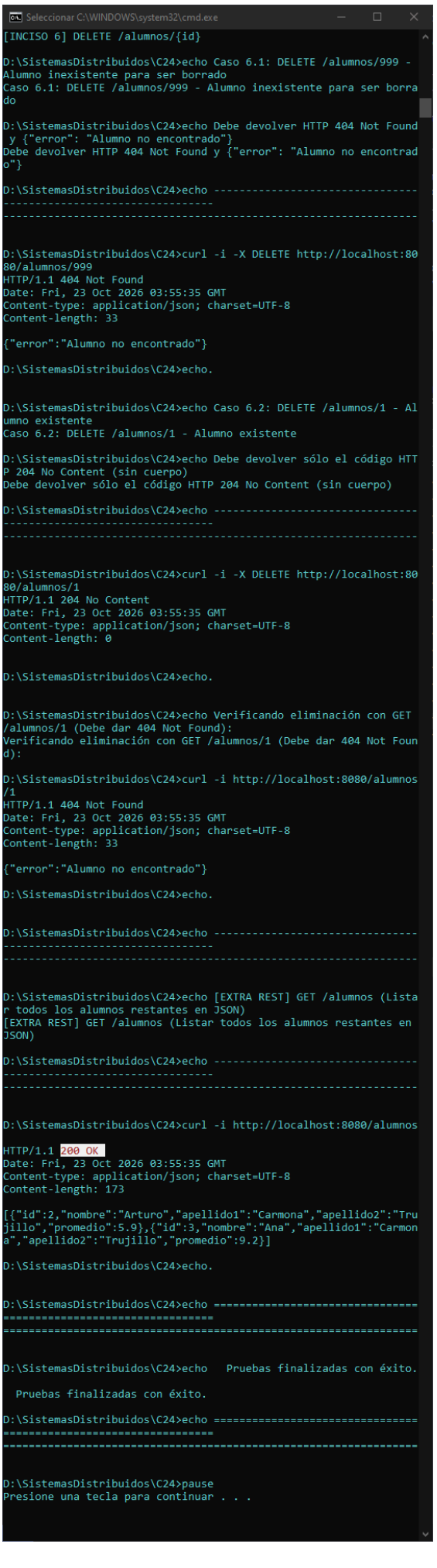
D:\SistemasDistribuidos\C24>echo [INCISO 1] Petición a endpoint inexistente o incorrecto (/chetito)
[INCISO 1] Petición a endpoint inexistente o incorrecto (/chetito)
D:\SistemasDistribuidos\C24>echo Debe devolver HTTP 404 Not Found y JSON de error
Debe devolver HTTP 404 Not Found y JSON de error
D:\SistemasDistribuidos\C24>echo ----------

D:\SistemasDistribuidos\C24>curl -i http://localhost:8080/chetito
HTTP/1.1 404 Not Found
Date: Fri, 23 Oct 2026 03:55:34 GMT
Content-type: application/json; charset=UTF-8
Content-length: 34
{"error":"Recurso no encontrado"}
D:\SistemasDistribuidos\C24>echo.
D:\SistemasDistribuidos\C24>echo ----------

D:\SistemasDistribuidos\C24>echo [INCISO 2] Peticiones con identificadores no numéricos
[INCISO 2] Peticiones con identificadores no numéricos
D:\SistemasDistribuidos\C24>echo Caso 2.1: GET /alumnos/p (Debe devolver HTTP 400 y {"error": "ID invalido"})
Caso 2.1: GET /alumnos/p (Debe devolver HTTP 400 y {"error": "ID invalido"})
D:\SistemasDistribuidos\C24>echo ----------

D:\SistemasDistribuidos\C24>curl -i http://localhost:8080/alumnos/p
HTTP/1.1 400 Bad Request
Date: Fri, 23 Oct 2026 03:55:34 GMT
Content-type: application/json; charset=UTF-8
Content-length: 24
{"error":"ID invalido"}
D:\SistemasDistribuidos\C24>echo.
D:\SistemasDistribuidos\C24>echo Caso 2.2: DELETE /alumnos/f (Debe devolver HTTP 400 y {"error": "ID invalido"})
Caso 2.2: DELETE /alumnos/f (Debe devolver HTTP 400 y {"error": "ID invalido"})
D:\SistemasDistribuidos\C24>curl -i -X DELETE http://localhost:8080/alumnos/f
HTTP/1.1 400 Bad Request
Date: Fri, 23 Oct 2026 03:55:34 GMT
Content-type: application/json; charset=UTF-8
Content-length: 24
{"error":"ID invalido"}
D:\SistemasDistribuidos\C24>echo.
D:\SistemasDistribuidos\C24>echo ----------

D:\SistemasDistribuidos\C24>echo [INCISO 3] POST /alumnos con cuerpo en formato JSON
[INCISO 3] POST /alumnos con cuerpo en formato JSON
D:\SistemasDistribuidos\C24>echo Se envían datos de Carlos Pérez Castro (promedio 8.7)


Seleccionar C:\WINDOWS\system32\cmd.exe
[INCISO 6] DELETE /alumnos/{id}
D:\SistemasDistribuidos\C24>echo Caso 6.1: DELETE /alumnos/999 - Alumno inexistente para ser borrado
Caso 6.1: DELETE /alumnos/999 - Alumno inexistente para ser borrado
D:\SistemasDistribuidos\C24>echo Debe devolver HTTP 404 Not Found y {"error": "Alumno no encontrado"}
Debe devolver HTTP 404 Not Found y {"error": "Alumno no encontrado"}
D:\SistemasDistribuidos\C24>echo ----------

D:\SistemasDistribuidos\C24>curl -i -X DELETE http://localhost:8080/alumnos/999
HTTP/1.1 404 Not Found
Date: Fri, 23 Oct 2026 03:55:35 GMT
Content-type: application/json; charset=UTF-8
Content-length: 33
{"error":"Alumno no encontrado"}
D:\SistemasDistribuidos\C24>echo.
D:\SistemasDistribuidos\C24>echo Caso 6.2: DELETE /alumnos/1 - Alumno existente
Caso 6.2: DELETE /alumnos/1 - Alumno existente
D:\SistemasDistribuidos\C24>echo Debe devolver sólo el código HTTP 204 No Content (sin cuerpo)
Debe devolver sólo el código HTTP 204 No Content (sin cuerpo)
D:\SistemasDistribuidos\C24>echo ----------

D:\SistemasDistribuidos\C24>curl -i -X DELETE http://localhost:8080/alumnos/1
HTTP/1.1 204 No Content
Date: Fri, 23 Oct 2026 03:55:35 GMT
Content-type: application/json; charset=UTF-8
Content-length: 0
D:\SistemasDistribuidos\C24>echo.
D:\SistemasDistribuidos\C24>echo Verificando eliminación con GET /alumnos/1 (Debe dar 404 Not Found):
Verificando eliminación con GET /alumnos/1 (Debe dar 404 Not Found):
D:\SistemasDistribuidos\C24>curl -i http://localhost:8080/alumnos/1
HTTP/1.1 404 Not Found
Date: Fri, 23 Oct 2026 03:55:35 GMT
Content-type: application/json; charset=UTF-8
Content-length: 33
{"error":"Alumno no encontrado"}
D:\SistemasDistribuidos\C24>echo.
D:\SistemasDistribuidos\C24>echo ----------

D:\SistemasDistribuidos\C24>echo [EXTRA REST] GET /alumnos (Listar todos los alumnos restantes en JSON)
[EXTRA REST] GET /alumnos (Listar todos los alumnos restantes en JSON)
D:\SistemasDistribuidos\C24>echo ----------

D:\SistemasDistribuidos\C24>curl -i http://localhost:8080/alumnos
HTTP/1.1 200 OK
Date: Fri, 23 Oct 2026 03:55:35 GMT
Content-type: application/json; charset=UTF-8
Content-length: 173
[{"id":2,"nombre":"Arturo","apellido1":"Carmona","apellido2":"Trujillo","promedio":5.9},{"id":3,"nombre":"Ana","apellido1":"Carmona","apellido2":"Trujillo","promedio":9.2}]
D:\SistemasDistribuidos\C24>echo.
D:\SistemasDistribuidos\C24>echo ==========
==========
==========
D:\SistemasDistribuidos\C24>echo Pruebas finalizadas con éxito.
Pruebas finalizadas con éxito.
D:\SistemasDistribuidos\C24>echo ==========
==========
==========
D:\SistemasDistribuidos\C24>pause
Presione una tecla para continuar . . .

```
Seleccionar C:\WINDOWS\system32\cmd.exe

D:\SistemasDistribuidos\C24>echo [INCISO 3] POST /alumnos con cuerpo en formato JSON
[INCISO 3] POST /alumnos con cuerpo en formato JSON

D:\SistemasDistribuidos\C24>echo Se envían datos de Carlos Pérez Castro (promedio 8.7)
Se envían datos de Carlos Pérez Castro (promedio 8.7)

D:\SistemasDistribuidos\C24>echo Debe devolver HTTP 201 Created y el JSON del alumno con su nuevo ID
Debe devolver HTTP 201 Created y el JSON del alumno con su nuevo ID

D:\SistemasDistribuidos\C24>echo ------------------------------------------------------------
------------------------------------------------------------

D:\SistemasDistribuidos\C24>curl -i -X POST -H "Content-Type: application/json" -d "{\"nombre\":\"Carlos\",\"apellido1\":\"Perez\",\
"apellido2\":\"Castro\",\"promedio\":8.7}" http://localhost:8080/alumnos
HTTP/1.1 201 Created
Date: Fri, 23 Oct 2026 03:55:34 GMT
Content-type: application/json; charset=UTF-8
Content-length: 83

{"id":1,"nombre":"Carlos","apellido1":"Perez","apellido2":"Castro","promedio":8.7}

D:\SistemasDistribuidos\C24>echo.


D:\SistemasDistribuidos\C24>echo Insertando alumnos adicionales para pruebas posteriores:
Insertando alumnos adicionales para pruebas posteriores:

D:\SistemasDistribuidos\C24>echo Insertando alumno 2 (Arturo Carmona Trujillo, 5.9):
Insertando alumno 2 (Arturo Carmona Trujillo, 5.9):

D:\SistemasDistribuidos\C24>curl -i -X POST -H "Content-Type: application/json" -d "{\"nombre\":\"Arturo\",\"apellido1\":\"Carmona\"
,\"apellido2\":\"Trujillo\",\"promedio\":5.9}" http://localhost:8080/alumnos
HTTP/1.1 201 Created
Date: Fri, 23 Oct 2026 03:55:34 GMT
Content-type: application/json; charset=UTF-8
Content-length: 87

{"id":2,"nombre":"Arturo","apellido1":"Carmona","apellido2":"Trujillo","promedio":5.9}

D:\SistemasDistribuidos\C24>echo.


D:\SistemasDistribuidos\C24>echo Insertando alumno 3 (Ana Carmona Trujillo, 9.2):
Insertando alumno 3 (Ana Carmona Trujillo, 9.2):

D:\SistemasDistribuidos\C24>curl -i -X POST -H "Content-Type: application/json" -d "{\"nombre\":\"Ana\",\"apellido1\":\"Carmona\",\"
apellido2\":\"Trujillo\",\"promedio\":9.2}" http://localhost:8080/alumnos
HTTP/1.1 201 Created
Date: Fri, 23 Oct 2026 03:55:34 GMT
Content-type: application/json; charset=UTF-8
Content-length: 84

{"id":3,"nombre":"Ana","apellido1":"Carmona","apellido2":"Trujillo","promedio":9.2}

D:\SistemasDistribuidos\C24>echo.


D:\SistemasDistribuidos\C24>echo ------------------------------------------------------------------
------------------------------------------------------------------

D:\SistemasDistribuidos\C24>echo [INCISO 4] GET /alumnos/{id}
[INCISO 4] GET /alumnos/{id}

D:\SistemasDistribuidos\C24>echo Caso 4.1: GET /alumnos/3 - Alumno existente
Caso 4.1: GET /alumnos/3 - Alumno existente

D:\SistemasDistribuidos\C24>echo Debe devolver HTTP 200 OK y el JSON con id, nombre, apellidos y promedio
Debe devolver HTTP 200 OK y el JSON con id, nombre, apellidos y promedio

D:\SistemasDistribuidos\C24>echo ------------------------------------------------------------------
------------------------------------------------------------------

D:\SistemasDistribuidos\C24>curl -i http://localhost:8080/alumnos/3
HTTP/1.1 200 OK
Date: Fri, 23 Oct 2026 03:55:34 GMT
Content-type: application/json; charset=UTF-8
Content-length: 84

{"id":3,"nombre":"Ana","apellido1":"Carmona","apellido2":"Trujillo","promedio":9.2}

D:\SistemasDistribuidos\C24>echo.


D:\SistemasDistribuidos\C24>echo Caso 4.2: GET /alumnos/666 - Alumno inexistente
Caso 4.2: GET /alumnos/666 - Alumno inexistente

D:\SistemasDistribuidos\C24>echo Debe devolver HTTP 404 Not Found y {"error": "Alumno no encontrado"}
Debe devolver HTTP 404 Not Found y {"error": "Alumno no encontrado"}

D:\SistemasDistribuidos\C24>echo ------------------------------------------------------------------
------------------------------------------------------------------

D:\SistemasDistribuidos\C24>curl -i http://localhost:8080/alumnos/666
HTTP/1.1 404 Not Found
Date: Fri, 23 Oct 2026 03:55:34 GMT
Content-type: application/json; charset=UTF-8
Content-length: 33

{"error":"Alumno no encontrado"}

D:\SistemasDistribuidos\C24>echo.


D:\SistemasDistribuidos\C24>echo ------------------------------------------------------------------
------------------------------------------------------------------

D:\SistemasDistribuidos\C24>echo [INCISO 5] PUT /alumnos/{id} con cuerpo en formato JSON
[INCISO 5] PUT /alumnos/{id} con cuerpo en formato JSON

D:\SistemasDistribuidos\C24>echo Caso 5.1: PUT /alumnos/1 - Actualización de alumno existente a promedio 9.2
Caso 5.1: PUT /alumnos/1 - Actualización de alumno existente a promedio 9.2

D:\SistemasDistribuidos\C24>echo Debe devolver HTTP 200 OK y {"mensaje": "Alumno actualizado"}
Debe devolver HTTP 200 OK y {"mensaje": "Alumno actualizado"}

D:\SistemasDistribuidos\C24>echo ------------------------------------------------------------------
------------------------------------------------------------------
```

```
C:\WINDOWS\system32\cmd.exe

D:\SistemasDistribuidos\C24>nฦๅ@echo off
"nฦๅ@echo" no se reconoce como un comando interno o externo,
programa o archivo por lotes ejecutable.

D:\SistemasDistribuidos\C24>chcp 65001  1>nul

D:\SistemasDistribuidos\C24>echo ==============================
==============================
================================================================


D:\SistemasDistribuidos\C24>echo   Iniciando Servidor RESTful mic
ro-SAES (Clase 24 - Puerto 8080)
  Iniciando Servidor RESTful micro-SAES (Clase 24 - Puerto 8080)

D:\SistemasDistribuidos\C24>echo ==============================
==============================
================================================================


D:\SistemasDistribuidos\C24>rem Verificar si el puerto 8080 ya es
tá ocupado y liberarlo automáticamente

D:\SistemasDistribuidos\C24>for /F "tokens=5" %a in ('netstat -ao
n | findstr :8080 | findstr LISTENING') do (
echo [AVISO] Puerto 8080 ocupado por proceso PID: %a. Liberando..
.
 taskkill /F /PID %a  1>nul 2>&1
 timeout /t 1  1>nul
)

D:\SistemasDistribuidos\C24>java -Dfile.encoding=UTF-8 -cp ".;gso
n-2.14.0.jar" WebServer 8080
================================================================

 Servidor RESTful micro-SAES (Clase 24 - Gson) iniciado en: 8080
 Endpoint de estado:  http://localhost:8080/status
 Endpoint de alumnos: http://localhost:8080/alumnos
 Formato de datos:    JSON (application/json)
================================================================

----------------------------------------------------------------

[ROOT HANDLER] Método: GET | Ruta: /chetito
[RESPUESTA 404] Endpoint inexistente o incorrecto: /chetito
----------------------------------------------------------------

[PETICIÓN] Método: GET | Ruta: /alumnos/p
[RESPUESTA 400] Identificador no numérico: p
----------------------------------------------------------------

[PETICIÓN] Método: DELETE | Ruta: /alumnos/f
[RESPUESTA 400] ID no numérico en DELETE: f
----------------------------------------------------------------

[PETICIÓN] Método: POST | Ruta: /alumnos
[RESPUESTA 201] Creado nuevo Alumno ID=1 | {"id":1,"nombre":"Carl
os","apellido1":"Perez","apellido2":"Castro","promedio":8.7}
----------------------------------------------------------------

[PETICIÓN] Método: POST | Ruta: /alumnos
[RESPUESTA 201] Creado nuevo Alumno ID=2 | {"id":2,"nombre":"Artu
ro","apellido1":"Carmona","apellido2":"Trujillo","promedio":5.9}
----------------------------------------------------------------

[PETICIÓN] Método: POST | Ruta: /alumnos
[RESPUESTA 201] Creado nuevo Alumno ID=3 | {"id":3,"nombre":"Ana"
,"apellido1":"Carmona","apellido2":"Trujillo","promedio":9.2}
----------------------------------------------------------------

[PETICIÓN] Método: GET | Ruta: /alumnos/3
[RESPUESTA 200] Alumno encontrado ID=3 | {"id":3,"nombre":"Ana","
apellido1":"Carmona","apellido2":"Trujillo","promedio":9.2}
----------------------------------------------------------------

[PETICIÓN] Método: GET | Ruta: /alumnos/666
[RESPUESTA 404] Alumno no encontrado ID=666
----------------------------------------------------------------

[PETICIÓN] Método: PUT | Ruta: /alumnos/1
[RESPUESTA 200] Alumno ID=1 actualizado | {"mensaje":"Alumno actu
alizado"}
----------------------------------------------------------------

[PETICIÓN] Método: GET | Ruta: /alumnos/1
[RESPUESTA 200] Alumno encontrado ID=1 | {"id":1,"nombre":"Carlos
","apellido1":"Perez","apellido2":"Castro","promedio":9.2}
----------------------------------------------------------------

[PETICIÓN] Método: PUT | Ruta: /alumnos/999
[RESPUESTA 404] Alumno no encontrado para actualizar ID=999
----------------------------------------------------------------

[PETICIÓN] Método: DELETE | Ruta: /alumnos/999
[RESPUESTA 404] Alumno no encontrado para eliminar ID=999
----------------------------------------------------------------

[PETICIÓN] Método: DELETE | Ruta: /alumnos/1
[RESPUESTA 204] Alumno ID=1 eliminado exitosamente
----------------------------------------------------------------

[PETICIÓN] Método: GET | Ruta: /alumnos/1
[RESPUESTA 404] Alumno no encontrado ID=1
----------------------------------------------------------------

[PETICIÓN] Método: GET | Ruta: /alumnos
[RESPUESTA 200] Listando 2 alumno(s)
```